

Communication

Date	8 July 2026
Team ID	XXXXXX
Project Name	Emotion Detection & Learning Support Engine
Maximum Marks	1 Mark

Communication Plan:

Effective communication is essential for a successful project demonstration. This document outlines the communication strategy used within the team and with stakeholders throughout the project lifecycle, including how updates, issues, and feedback were managed.

S.No	Communication Type	Frequency	Channel / Tool	Participants	Purpose
1	Personal Standup & Planning	Daily	Notion / Trello	Self	Self-tracking progress: reviewing yesterday's completed tasks, setting today's goals, and noting immediate blockers.
2	Progress Update	Weekly	Email / WhatsApp	Self & Faculty/Mentor	Providing a high-level summary of milestone completions (e.g., model training, UI setup) and discussing next steps.
3	Issue / Bug Discussion	As Needed	GitHub Issues / WhatsApp	Self & Developer Community	Documenting and troubleshooting technical errors (e.g., API limits, TensorFlow bugs) using StackOverflow or GitHub Discussions.
4	Stakeholder Review	Bi-Weekly	Google Meet / Email	Self & Faculty/Mentor	Presenting working prototypes to get external feedback on UI/UX and the quality of the AI-generated responses
5	Final Demo Rehearsal	Once	Google Meet	Self	Performing a solo dry-run of the presentation, recording it to review timing, flow, and live code execution.
6	Code Reviews	As Needed	GitHub Pull Requests	Self	Using detailed commit messages as a form of communication to track code changes and logic updates.

Communication Challenges & Resolutions:

S.No	Challenge Faced	Resolution / Action Taken
1	Lack of Peer Feedback: Working solo meant there was no immediate peer available for code reviews or bouncing architectural ideas off of.	Action Taken: Leveraged AI coding assistants as a "pair-programmer" for rapid feedback, and scheduled bi-weekly Google Meets with my faculty mentor to ensure the project stayed on track.
2	Technical Blockers & Isolation: Encountering complex bugs (like integrating the BiLSTM model with Streamlit) without a team to split the debugging workload.	Action Taken: Adopted a highly organized issue-tracking system on GitHub. Broke large bugs down into smaller, searchable errors and utilized developer communities (StackOverflow/Discord) to resolve them efficiently.
3	Time Management & Scope Creep: Without a project manager, it was challenging to communicate to myself which features to prioritize versus which ones to cut.	Action Taken: Implemented a strict Kanban board (Trello/Notion). Communicated priority shifts clearly by moving non-essential features (like extra animations) to a "Backlog" column to focus purely on the core ML and API integration.